AgileGraph

Expert Reviewer Labelling Guide

Independent Post-Quantum Cryptographic Risk Classification Study

Institution: NMAM Institute of Technology (NMAMIT), Nitte, Karnataka

Project Guide: Ms. Madhushree, Asst. Professor, Dept. of CSE (Cyber Security)

Study Type: Blinded independent expert validation — do NOT consult other reviewers

Time Commitment: Approximately 30–45 minutes

1. About This Study

AgileGraph is a final-year B.Tech Major Project that constructs a dependency graph of software repositories and applies a Graph Attention Network (GATv2) to score cryptographic assets by their post-quantum migration risk. To scientifically validate the model's output, we are recruiting independent cybersecurity professionals to classify a curated set of cryptographic source code files **without seeing the model's scores**. Your assessments will be compared against the model using inter-annotator agreement metrics (Weighted Cohen's Kappa, Fleiss' Kappa) and classification metrics (Macro-F1 with bootstrap confidence intervals).

■ **Note:** Your labels will **never** be used to train or fine-tune the model. They are used exclusively for post-hoc evaluation and validation.

2. Files You Received

File	Description	What to do
AgileGraph_Review_Sheet.xlsx	160 cryptographic source files, one per row. Columns A–D are pre-filled context. You fill columns E, F, and G.	Open in Excel or Google Sheets. Fill columns E–G only.
AgileGraph_Labelling_Guide.pdf	This document — rubric, examples, step-by-step instructions.	Read before starting. Keep open as a reference.

3. Review Sheet Column Guide

Column	Label	Filled by	What it contains / What to enter
A	Asset ID	Pre-filled	Source file path within the repository (read-only context).
B	Repository	Pre-filled	Benchmark repository name (e.g., go_crypto, python_cryptography).
C	Language	Pre-filled	Programming language of the file: Go, Python, or Java.
D	Detected Cryptographic Primitives	Pre-filled	Automatically detected cryptographic API calls and algorithms found in the file (e.g., RSA-2048, AES-128). Use this as your primary evidence.
E	**Expert Assigned Risk**	**YOU FILL**	**Select from dropdown: HIGH, MEDIUM, or LOW.** See Section 4 for definitions.
F	Expert Confidence	YOU FILL	Enter a decimal from 0.0 (completely uncertain) to 1.0 (absolutely certain). E.g., 0.9

G	Expert Rationale	YOU FILL	1–2 sentences explaining your classification. E.g.: "RSA-2048 is used for key exchange and is vulnerable to Shor's algorithm on a CRQC."

4. Risk Classification Rubric

Classify each asset into exactly one of the three categories below, based on the detected cryptographic primitives and their known vulnerabilities to quantum and classical attacks.

■ HIGH

Immediate Migration Priority

- RSA (all key lengths: RSA-1024, RSA-2048, RSA-4096) — factorization broken by Shor's algorithm
- ECC, ECDSA, ECDH, Ed25519, X25519, secp256k1, P-256, P-384, P-521 — discrete log broken
- DSA, Diffie-Hellman — discrete log problem solved on a CRQC
- MD5, SHA-1 in security paths (signature verification, integrity) — classically broken
- DES, 3DES, RC4, Blowfish — classically broken or weak
- Any long-lived sensitive data protected by the above (Harvest Now, Decrypt Later risk)

■ MEDIUM

Scheduled Migration / Defense-in-Depth

- AES-128 — Grover's algorithm reduces effective security from 128 to ~64 bits; migration to AES-256 recommended
- Legacy block ciphers with marginal key lengths in constrained internal environments
- Transitional / hybrid deployments (partial PQC encapsulation, not fully quantum-safe)
- High-centrality cryptographic abstraction layers whose replacement requires systemic refactoring

■ LOW

Post-Quantum Resilient / No Immediate Action Required

- NIST FIPS 203: ML-KEM / Kyber — standardized post-quantum key encapsulation
- NIST FIPS 204: ML-DSA / Dilithium — standardized post-quantum digital signature
- NIST FIPS 205: SLH-DSA / SPHINCS+ — standardized post-quantum signature
- AES-256, ChaCha20-Poly1305 — quantum-resistant symmetric encryption (128-bit quantum security retained)
- SHA-256, SHA-384, SHA-512, SHA-3, HMAC (≥256 bits) — quantum-resistant hash and MAC
- Hash functions used only for non-adversarial purposes (caching, hash maps, checksums — not authentication)

5. Step-by-Step Instructions

Step 1 — Fill in your reviewer profile	In the Review Sheet, go to Sheet 1 "Reviewer Profile & Rubric". Fill in cells B5-B9: • Name / Pseudonym (B5), Years of Experience (B6), Professional Role (B7), Affiliation (B8), Date (B9).

Step 2 — Go to the Asset Review sheet	Click the "Asset Review (160)" tab. Each row is one source code file. Read columns A-D for context: file path, repository, language, and detected primitives.
Step 3 — Classify each asset (Column E)	Click the cell in column E for that row. A dropdown will appear. Select HIGH, MEDIUM, or LOW. Base your decision on the cryptographic primitive in column D and the rubric in Section 4 of this guide.
Step 4 — Enter your confidence (Column F)	Type a decimal between 0.0 and 1.0. 0.0 = completely uncertain. | 0.5 = moderately confident. | 1.0 = absolutely certain.
Step 5 — Write a brief rationale (Column G)	Write 1–2 sentences explaining your classification. Example: "RSA-2048 is used for key exchange; vulnerable to Shor's algorithm on a CRQC. Immediate migration to ML-KEM required."
Step 6 — Save and return	Save the file as Excel (.xlsx). Reply to the invitation email with the completed file attached. Do not share your answers with other reviewers before submission.

6. Worked Classification Examples

Asset (abbreviated)	Language	Detected Primitives	Risk	Confidence	Sample Rationale
rsa.py python_cryptography	Python	RSA-2048, RSAPrivateKey	HIGH	0.95	RSA-2048 is broken in polynomial time by Shor's algorithm on a CRQC; immediate migration to ML-KEM required.
jws.go go_crypto	Go	hmac.New(), sha256.Sum256()	LOW	0.90	HMAC-SHA256 retains 128-bit quantum security; Grover's algorithm does not reduce it below safe margins.
aes.py python_cryptography	Python	AES-128	MEDIUM	0.80	AES-128 effective security is halved to ~64 bits by Grover's algorithm; migration to AES-256 is recommended.
EcKeySignerFactory.java java_jjwt	Java	ECDSA, P-256, SHA256withECDSA	HIGH	0.95	ECDSA with P-256 is broken by Shor's algorithm on a CRQC; immediate migration to ML-DSA required.

7. Frequently Asked Questions

Q: Do I need to read the actual source code?	A: No. Column D already shows the detected cryptographic primitives. You only need to classify based on those primitives. However, if you want to verify or supplement, the repository name and file path in columns A–B indicate the source.
Q: What if I am unsure about a primitive?	A: Assign the risk level you lean towards and reduce your confidence score (column F) accordingly. A confidence of 0.5 is perfectly acceptable for ambiguous cases.
Q: What if a file has multiple primitives of different risk levels?	A: Assign the highest applicable risk level. A file using both RSA-2048 (HIGH) and SHA-256 (LOW) should be classified HIGH.

Q: Can I skip rows I am not confident about?	A: Please try to classify all 160 rows. If you truly cannot assess a particular file, leave columns E–G blank for that row only and add a note in column G saying "Insufficient context".
Q: Will I see the model's scores?	A: No. This is a blinded study. The model's risk scores are hidden to prevent bias. Your independent assessment is the scientific value of this exercise.
Q: How do I return my completed sheet?	A: Save the file as Excel (.xlsx) and reply to the invitation email with the file attached. Do not upload it to any shared drive or forward it to other reviewers before submission.